

ASPERITAS Biological Intelligence OS

AI 에이전트 최종 목표, Phase별 개발 로드맵, 벤치마크 통합형 최상위 프로세스

Version: 2026-06-27 / Strategy PDF

Scope: Asperitas RAG Agent를 기반으로 생물다양성, 합성생물학, DBTL, 바이오파운드리, Foundation Model for Biology까지 연결하는 내부 AI 운영체제 설계

Bottom Line

지금 만드는 것은 단순 RAG 챗봇이 아니다. Asperitas의 미래 Biology Foundation Model을 가능하게 할 데이터, 지식, 실험, 규제, IP 운영체제의 1단계 코어다. 최종 목표는 생물다양성 접근권과 독자 실험 데이터를 AI가 읽고, 연결하고, 실험을 설계하고, 실패를 학습하며, DBTL 사이클을 가속하는 내부 Biological Intelligence OS다.

Biodiversity Access

- > Governed Data Registry
- > Source-grounded RAG / Knowledge Graph
- > Research, Design, Compliance Agents
- > DBTL Loop: Design -> Build -> Test -> Learn
- > Biofoundry Ops / Active Learning
- > Proprietary Biological Dataset
- > Asperitas Foundation Model for Biology

작성 기준: 공개 공식 자료와 신뢰 가능한 기술 문서를 벤치마킹하되, Asperitas의 목적에 맞게 재조합했다. 이 문서는 실행 설계서이며, 특정 외부 회사의 내부 프로세스를 복제했다는 의미가 아니다.

1. 벤치마크에서 추출한 핵심 원칙

아래 표는 회사별 공개 자료에서 확인 가능한 운영 원칙을 Asperitas 개발 원칙으로 변환한 것이다. 핵심은 '좋아 보이는 기능을 다 बे끼는 것'이 아니라, 과학 AI 운영체계에 필요한 장점만 가져와 하나의 엄격한 프로세스로 통합하는 것이다.

벤치마크	공개 자료에서 보이는 강점	Asperitas에 흡수할 원칙
OpenAI	Agents SDK는 planning, tool use, specialist handoff, state를 가진 agent 구조를 제시한다. Agent safety 문서는 guardrails, tool approval, trace grading, evals를 강조한다. [S1] [S2]	모든 에이전트는 trace, eval, guardrail, approval을 기본값으로 둔다. 데모 성공이 아니라 eval 통과가 merge 조건이다.
Anthropic	효과적인 agent는 복잡한 프레임워크보다 simple, composable pattern이 잘 작동한다고 설명하며 workflows와 agents를 구분한다. RSP는 capability threshold와 ASL식 safeguard 강화를 제시한다. [S3] [S4]	처음부터 agent swarm을 만들지 않는다. 먼저 명확한 workflow를 만들고, 불확실성과 위험이 커지는 지점만 agent autonomy로 확장한다.
Google DeepMind / Isomorphic	AlphaFold 이후 digital biology와 predictive/generative AI models를 통해 drug discovery를 가속하는 방향을 제시한다. [S10]	RAG의 최종 목적은 문서 요약이 아니라, 검증된 생물학적 설계 능력과 foundation-model-ready 데이터 생산이다.
Recursion	Recursion OS는 대규모 proprietary biological/chemical dataset, automated high-throughput lab, maps of biology/chemistry, active learning 기반 molecule selection을 강조한다. [S8]	Asperitas의 moat는 논문 DB가 아니라 독자 실험 데이터, 실패 이력, perturbation/phenotype 연결 데이터, active learning loop다.
Ginkgo Bioworks	Ginkgo는 labwork bottleneck을 autonomous labs로 풀고, cloud lab을 통해 실험 목표에 맞춘 디자인을 지원한다고 설명한다. [S7]	wet-lab 반복작업을 자동화 가능한 unit operation으로 쪼개고, AI는 과학자가 insight와 judgment에 집중하게 만든다.
Benchling	R&D Cloud는 data entry, registration, tracking, connected searchable platform, compliant scalable workflows를 강조한다. [S6]	모든 자료, 샘플, 프로토콜, 실험결과, 버전은 searchable registry로 남긴다. 파일 더미가 아니라 운영 DB가 되어야 한다.
LangGraph	long-running stateful workflow, persistence, human-in-the-loop, memory, debugging visibility를 제공한다. [S5]	장기 실행, 승인 대기, 실패 복구, 상태 추적이 필요한 에이전트는 state machine/graph로 관리한다.
DBTL / Biofoundry literature	합성생물학은 Design-Build-Test-Learn cycle을 통해 설계, 제작, 테스트, 학습을 반복하며 AI와 automated labs가 이를 가속한다. [S9]	Asperitas의 모든 연구 에이전트는 DBTL loop에 귀속되어야 한다. 답변이 아니라 다음 cycle의 품질 향상이 목표다.

2. Asperitas 독자 최상위 프로세스: A-GRID

A-GRID는 Asperitas Governed Research Intelligence DBTL의 약자다. 공개 벤치마크의 장점을 합쳐 만든 내부 표준 프로세스이며, 모든 AI 에이전트 개발과 연구 운영을 이 루프에 넣는다.

A-GRID Loop

1. Govern : source, license, biosafety, IP, approval 상태를 먼저 잠근다.
2. Retrieve : 필요한 근거만 검색하고, broad retrieval -> rerank -> top-K compression을 적용한다.
3. Interpret : 근거 기반으로 요약, 비교, gap, uncertainty를 분리한다.
4. Design : hypothesis, experiment, protocol, pathway 후보를 설계한다.
5. Validate : eval, compliance, biosafety, human approval gate를 통과한다.
6. Execute : approved plan만 build/test 또는 dry-run/simulation으로 실행한다.
7. Learn : 결과, 실패, metadata, metrics를 다시 DB화한다.
8. Compound : 다음 cycle에서 더 나은 데이터와 모델로 누적 개선한다.

A-GRID의 핵심 원칙

- 승인되지 않은 자료는 절대 ingestion하지 않는다. raw_available과 approved_ingested를 분리한다.
- 추론 품질을 줄이지 않고, 쓸모없는 context만 줄인다. 이것이 v1.4 token minimization의 핵심이다.
- 모든 산출물은 source_id, filename, registry key, version, status, number, verification command를 보존한다.
- agent autonomy는 목적이 아니다. 반복 가능한 workflow가 먼저이고, autonomy는 병목 구간에 제한적으로 붙인다.
- 모든 실험형 agent는 biosafety, compliance, IP, human approval gate 뒤에 둔다.
- 성능 향상은 느낌이 아니라 eval로 판단한다: retrieval hit, citation fidelity, answer accuracy, latency, token cost, regression 여부를 측정한다.

이 프로세스의 최종 출력은 '답변'이 아니라 '더 나은 DBTL cycle'이다. 즉, 모든 AI agent는 새로운 지식, 검증 가능한 근거, 실험 우선순위, 실패 학습, 독자 데이터 축적으로 이어져야 한다.

Phase 전체 개요

Phase	목표	핵심 Agent	Gate
0. Core RAG & Governance	근거 기반 기억장치와 승인형 DB화 체계 확립	Source Registry, Ingestion, Retrieval Eval, Grounded Answer, Compliance Router	approved-only ingestion, eval pass, citation fidelity
1. Intelligence Acquisition	외부 논문/기업/규제/벤치마크 탐색 자동화	Deep Research, Literature, Benchmark, Competitive Intelligence	license, source quality, duplication, relevance
2. Scientific Reasoning	가설, mechanism, design option 생성	Hypothesis, Mechanism, Design Review	evidence trace, uncertainty, falsifiability
3. Experiment & Protocol	실험 설계와 프로토콜 후보를 안전하게 구조화	Experiment Design, Protocol, Simulation Planning	biosafety, dual-use, human approval
4. DBTL Operations	Design-Build-Test-Learn 반복 운영	DBTL Planner, Failure Analysis, Biofoundry Ops	sample/protocol/result traceability
5. Active Learning & Data Flywheel	다음 실험의 정보가치를 극대화	Active Learning, Dataset Curator, Model Eval	data quality, model gain, experiment ROI
6. Foundation Model Readiness	독자 Biological Foundation Model 학습 기반 확보	Training Data Pipeline, Multimodal Bio Model Eval	proprietary data moat, safety eval, reproducibility
7. Platform & Business OS	IP, IR, grant, productization으로 사업화 연결	IP Scout, Licensing, IR, Grant/RFP, Onboarding	claim substantiation, compliance, commercialization fit

Phase 0 – Core RAG & Governance

목표: Asperitas AI의 장기 기억장치, 승인형 ingestion, 근거 기반 답변, 평가/컴플라이언스 코어를 완성한다.

구현 Agent

- Source Registry Agent
- Ingestion & Chunking Agent
- Retrieval Quality Agent
- Grounded Answer Agent
- Truth / Compliance Router

개발 과정

Step	개발 내용
1	source registry schema를 확정한다: source_id, title, source_type, license, approval_status, ingestion_status, owner, version, risk_level.
2	raw source와 processed source를 분리한다. raw가 있다고 DB화된 것으로 간주하지 않는다.
3	approved_only ingestion을 강제한다. needs_review, blocked, raw_available_not_ingested는 index에 넣지 않는다.
4	chunk metadata를 고정한다: source_id, file path, section, page, registry key, created_at, checksum.
5	retrieval eval set을 만든다. 각 질문마다 gold source, expected section, acceptable answer contract를 둔다.
6	answer generator는 retrieved context 밖의 내용을 추정하지 못하게 한다. 모르면 모른다고 답하게 한다.
7	모든 PR은 registry diff, chunk count, eval delta, test command, rollback rule을 포함한다.

평가지표

- registry validation pass
- chunk metadata coverage
- top-K retrieval hit rate
- citation fidelity
- hallucination zero tolerance
- token/input cost per answer
- latency

Done 기준: V1 내부 사용자가 질문했을 때, 승인된 자료만 근거로 짧고 정확하게 답하며, 어떤 파일/section을 보고 답했는지 추적 가능해야 한다.

Phase 1 – Intelligence Acquisition Layer

목표: 외부 지식 수집을 수동 검색에서 source-governed research pipeline으로 전환한다.

구현 Agent

- Deep Research Agent
- Literature Intelligence Agent
- Benchmark Intelligence Agent
- Competitive Intelligence Agent

개발 과정

Step	개발 내용
1	research question을 market, biology, technology, regulatory, competitor, IP로 분해한다.
2	공식 사이트, 논문, preprint, 특허, 규제기관, 뉴스의 source 후보를 수집한다.
3	각 source에 license, commercial-use, model-training 가능성, ingestion 가능 여부를 태깅한다.
4	중복 제거와 source ranking을 수행한다: authority, recency, relevance, reproducibility 기준.
5	registry 초안만 생성하고, human review 후 approved source만 Phase 0 ingestion으로 넘긴다.
6	벤치마크 회사별 workflow 장점을 internal playbook으로 변환한다.

평가지표

- source precision
- duplicate rate
- license resolved rate
- approved ingestion ratio
- research-to-registry turnaround
- benchmark actionability score

Done 기준: Deep Research 결과가 곧바로 답변으로 소비되는 것이 아니라, registry와 review를 거쳐 재사용 가능한 내부 지식 자산이 되어야 한다.

Phase 2 – Scientific Reasoning & Hypothesis Layer

목표: 자료 요약을 넘어서, 검증 가능한 생물학적 가설과 설계 후보를 생성한다.

구현 Agent

- Hypothesis Generation Agent
- Mechanism Mapping Agent
- Design Review Agent
- Evidence Gap Agent

개발 과정

Step	개발 내용
1	질문을 biological system, target function, organism/context, measurable output, constraints로 분해한다.
2	RAG로 근거를 검색하고 mechanism map을 만든다: entity, relation, condition, evidence strength.
3	가설은 falsifiable statement로 작성한다: 어떤 조건에서 어떤 결과가 나오면 지지/반박되는지 명시한다.
4	후보 설계안은 expected upside, uncertainty, cost, biosafety/IP risk로 랭킹한다.
5	evidence gap을 자동으로 기록해 다음 Deep Research 또는 실험 설계로 넘긴다.

평가지표

- hypothesis traceability
- falsifiability
- evidence coverage
- uncertainty clarity
- design option diversity
- reviewer acceptance

Done 기준: AI가 '그럴듯한 아이디어'를 내는 것이 아니라, 근거와 반증 기준이 있는 실험 후보를 만들어야 한다.

Phase 3 – Experiment & Protocol Layer

목표: 실험 설계, 프로토콜 후보, 시뮬레이션 계획을 safety-first 방식으로 구조화한다.

구현 Agent

- Experiment Design Agent
- Protocol Design Agent
- Simulation Planning Agent
- Biosafety Gate Agent

개발 과정

Step	개발 내용
1	실험 목적, 독립변수, 종속변수, control, replicate, readout, success criteria를 표준 양식으로 만든다.
2	protocol 후보는 안전 등급, 필요한 장비/시약, ambiguity, failure points를 태깅한다.
3	wet-lab 실행 가능성이 있는 내용은 biosafety, dual-use, regulatory, human approval gate를 통과해야 한다.
4	가능하면 먼저 dry-run/simulation 또는 literature-backed feasibility review를 수행한다.
5	실험 실행 전에는 sample ID, protocol version, operator, equipment, expected output을 lock한다.

평가지표

- protocol completeness
- risk classification accuracy
- human approval latency
- simulation-to-experiment agreement
- failure-prevention checklist coverage

Done 기준: 프로토콜이 생성되는 것이 아니라, 실행 전 승인 가능한 controlled experiment package가 생성되어야 한다.

Phase 4 – DBTL Operations Layer

목표: 실험을 일회성 이벤트가 아니라 반복 개선 루프로 운영한다.

구현 Agent

- DBTL Planning Agent
- Experiment Log Agent
- Failure Analysis Agent
- Biofoundry Ops Agent

개발 과정

Step	개발 내용
1	Design: 가설과 설계 후보를 cycle objective, design variables, predicted outcome으로 고정한다.
2	Build: construct/sample/protocol/equipment 정보를 registry화한다.
3	Test: readout, QC, measurement error, replicate consistency를 기록한다.
4	Learn: 성공/실패 원인을 feature, condition, protocol, biological mechanism 단위로 분류한다.
5	Next Design: 실패 데이터도 버리지 않고 다음 cycle의 active learning input으로 전환한다.

평가지표

- cycle time
- experiment throughput
- data completeness
- failure taxonomy coverage
- learn-to-next-design conversion
- reproducibility

Done 기준: 실험 결과가 PDF 보고서로 끝나는 것이 아니라, 다음 design decision을 개선하는 structured data로 축적되어야 한다.

Phase 5 – Active Learning & Proprietary Data Flywheel

목표: 다음 실험의 정보가치를 극대화하여 데이터 해자를 만든다.

구현 Agent

- Active Learning Agent
- Dataset Curator Agent
- Model Evaluation Agent
- Data Quality Gate Agent

개발 과정

Step	개발 내용
1	모델이 확신하는 영역과 불확실한 영역을 분리한다.
2	다음 실험 후보를 expected information gain, cost, risk, strategic moat 기준으로 랭킹한다.
3	데이터셋은 modality별로 정리한다: sequence, structure, phenotype, omics, assay, protocol, failure log.
4	모델 성능 개선이 실제 실험/설계 품질 개선으로 이어지는지 eval한다.
5	외부 공개 데이터와 Asperitas 독자 데이터는 엄격히 분리하여 데이터 권리와 모델 훈련 가능성을 관리한다.

평가지표

- information gain per experiment
- data quality score
- model improvement delta
- unique proprietary coverage
- cost per useful datapoint
- dataset audit pass

Done 기준: Asperitas만 가진 데이터가 축적되고, 이 데이터가 다음 실험 선택과 모델 성능을 동시에 개선해야 한다.

Phase 6 – Foundation Model for Biology Readiness

목표: 독자 Biological Foundation Model 학습이 가능한 데이터, 평가, 안전 체계를 준비한다.

구현 Agent

- Training Data Pipeline Agent
- Bio Model Eval Agent
- Model Card Agent
- Safety Red-Team Agent

개발 과정

Step	개발 내용
1	학습 가능한 데이터와 단순 검색용 자료를 분리한다. license/model-training permission을 명시한다.
2	pretraining, retrieval-augmented generation, fine-tuning, model distillation 중 무엇이 필요한지 use-case별로 결정한다.
3	foundation model 전에는 domain-specific evaluator를 먼저 만든다: biology QA, mechanism consistency, design plausibility, safety refusal.
4	모델 학습은 무조건 dataset version, training config, eval result, safety result를 남긴다.
5	biosecurity-sensitive capability는 capability threshold와 safeguard 기준을 적용한다.

평가지표

- trainable dataset volume/quality
- eval benchmark coverage
- model delta vs baseline
- safety eval pass
- reproducible training run
- license clean rate

Done 기준: Foundation Model은 구호가 아니라, 학습 가능한 독자 데이터와 평가 체계가 있을 때만 시작한다.

Phase 7 – Platform, IP & Business OS

목표: 연구 결과를 특허, 파트너십, IR, 정부과제, 제품 로드맵으로 연결한다.

구현 Agent

- IP / Patent Scout Agent
- Data Licensing Agent
- Investor / IR Agent
- Grant / RFP Agent
- Talent / Onboarding Agent

개발 과정

Step	개발 내용
1	DBTL 결과에서 patentable novelty, freedom-to-operate risk, trade secret 후보를 분리한다.
2	외부 발표, IR, grant 문서는 source-grounded claim만 허용한다.
3	제품화 후보는 market pain, technical moat, regulatory path, unit economics로 평가한다.
4	신규 팀원은 RAG/DBTL/source governance/on-call 운영규칙을 onboarding agent로 학습한다.
5	외부 공개 가능한 내용과 내부 비공개 내용을 자동 분리한다.

평가지표

- IP candidate count
- FTO risk clarity
- claim substantiation
- grant fit score
- onboarding time
- external disclosure risk

Done 기준: AI가 연구 보조를 넘어 회사의 실행속도, 자산화, 방어가능한 moat 형성에 직접 기여해야 한다.

3. 모든 Phase에 적용되는 개발 프로세스

각 Phase는 기능 아이디어 -> 구현 -> 테스트가 아니라, 아래의 반복 가능한 engineering loop로 개발한다. 이것이 Asperitas의 독자적 개발 공정이다.

Asperitas Agent Development Loop

0. Scope Lock
 - 이번 PR/phase에서 바꾸는 것과 절대 바꾸지 않는 것을 명시한다.
1. Source & Risk Preflight
 - 필요한 source, license, biosafety, compliance, IP risk를 먼저 확인한다.
2. Contract Design
 - input schema, output schema, stop rules, approval rules, eval rules를 작성한다.
3. Minimal Implementation
 - 가장 작은 vertical slice를 구현한다. 복잡한 agent보다 검증 가능한 workflow를 우선한다.
4. Eval Harness
 - unit tests, integration tests, retrieval evals, trace graders, safety tests를 붙인다.
5. Dry Run & Regression
 - 실제 ingestion/deploy 전 dry-run evidence와 diff를 남긴다.
6. Human Gate
 - ingestion, wet-lab, external claim, IP-sensitive output은 human approval을 통과한다.
7. Merge & Evidence Log
 - changed files, commands run, tests, eval deltas, source counts, skipped/blocked items를 기록한다.
8. Learn Back
 - failure taxonomy와 improvement backlog를 다음 phase의 input으로 넘긴다.

PR/작업 산출물 표준

항목	반드시 남길 내용
Executive Bottom Line	이번 작업이 GO인지 NO-GO인지, 왜 그런지 한 문단으로 정리
Scope	변경한 것 / 변경하지 않은 것 / 금지된 것
Files Changed	파일 경로, 목적, 영향 범위
Source Counts	raw, approved, ingested, skipped, blocked, needs_review
Commands Run	검증 가능한 shell command, pytest, registry check, eval run
Tests/Evals	pass/fail, regression 여부, metric delta
Compliance	license, biosafety, dual-use, privacy, IP 관련 결론
Next Actions	다음 PR에서 할 것. 현재 PR에서 하지 않은 것

토큰 최소화 규칙

- 컨텍스트를 줄이되 추론 품질은 줄이지 않는다.
- 전체 문서 dump 대신 broad retrieval -> rerank -> top-K narrowing -> query-focused compression을 사용한다.
- source_id, filename, registry key, version, status, numbers, verification commands, compliance constraints는 절대 제거하지 않는다.
- 출력은 고정 contract로 짧게 한다: Bottom Line, Evidence, Decision, Risks, Next Commands.
- negative scope와 stop rule을 명확히 적는다. 하지 말아야 할 일을 줄이면 토큰과 리스크가 동시에 줄어든다.

4. 구현 우선순위 - 지금부터의 실행 순서

현재 Asperitas RAG Agent 기준으로 가장 효율적인 순서는 아래다. 이 순서를 지키면 기능이 늘어나도 코어 안정성, source governance, eval discipline이 무너지지 않는다.

우선순위	작업	이유	완료 기준
P0	Source Registry / approved-only ingestion hardening	자료 오염과 license risk를 막는 최상위 gate	unapproved source가 index에 들어갈 수 없는 구조
P0	Retrieval eval + citation fidelity	RAG 품질은 답변 느낌이 아니라 근거 적중률로 판단	gold source hit, section-level citation, regression report
P0	Grounded answer contract	짧고 정확한 답변을 안정적으로 재현	모르면 모른다고 답하고, 근거 밖 추정 금지
P1	Deep Research -> Registry pipeline	외부 자료 수집을 DB화 가능한 workflow로 변환	source 후보가 review/approval/ingestion 상태로 분리
P1	Literature / Benchmark Agent	논문과 기업 workflow를 내부 playbook으로 변환	evidence-backed action item 생성
P2	Hypothesis / Experiment Design Agent	RAG를 연구 의사결정 도구로 확장	falsifiable hypothesis와 experiment package 생성
P2	Biosafety / Compliance Router	바이오 agent의 필수 안전장치	risk route, refusal, human approval path 검증
P3	DBTL Planner + Failure Analysis	실험 결과를 데이터 flywheel로 전환	failure taxonomy와 next-cycle recommendation
P4	Active Learning + Proprietary Dataset	Foundation Model의 해자 형성	정보가치 기반 실험 우선순위와 데이터셋 버전 관리

NO-GO 조건

- source registry가 불완전한데 외부 자료를 대량 ingestion하려는 경우
- eval 없이 prompt나 retrieval logic을 바꾸는 경우
- biosafety/compliance gate 없이 wet-lab protocol agent를 자동화하려는 경우
- 각 agent를 별도 repo로 성급히 분리하여 공통 memory/eval/compliance core가 깨지는 경우
- 외부 벤치마크 자료를 내부 독자 데이터와 같은 등급으로 취급하는 경우

핵심 판단: 지금은 agent swarm을 만드는 단계가 아니라, agent factory의 엔진을 만드는 단계다. 공통 RAG Core, Registry, Eval, Compliance가 안정화된 뒤 역할별 agent를 모듈로 분리한다.

5. 최종 Agent Map

Layer	Agent	Primary Job	핵심 데이터
Core Memory	Source Registry Agent	출처/라이선스/승인상태 관리	source registry, schema, approval log
Core Memory	Ingestion Agent	승인 자료 chunking/embedding/indexing	raw, processed, chunks, embeddings
Core Memory	Retrieval Eval Agent	검색 품질 측정	eval set, gold source, metrics
Core Memory	Grounded Answer Agent	근거 기반 답변	retrieved context, citations
Governance	Compliance Router	biosafety/license/IP/privacy gate	risk policy, approval state
Intelligence	Deep Research Agent	외부 자료 탐색과 registry 초안	papers, patents, official pages, news
Intelligence	Literature Agent	논문 method/result/limit 추출	scientific papers, notes
Intelligence	Benchmark Agent	회사 workflow 벤치마킹	company sources, playbooks
R&D	Hypothesis Agent	검증 가능한 가설 생성	mechanism map, evidence gaps
R&D	Experiment Design Agent	실험 변수/control/readout 설계	experiment package
R&D	Protocol Agent	프로토콜 후보와 위험점 구조화	protocol versions, SOP
DBTL	DBTL Planner	cycle 운영과 next-cycle 결정	design/build/test/learn logs
DBTL	Failure Analysis Agent	실패 원인 자산화	failure taxonomy
Biofoundry	Biofoundry Ops Agent	장비/샘플/스케줄 운영	LIMS, equipment, sample ID
Model	Active Learning Agent	정보가치 높은 실험 추천	uncertainty, model predictions
Model	Foundation Model Data Agent	학습 가능 데이터셋 버전 관리	trainable multimodal datasets
Business	IP / Patent Scout Agent	특허/FTO/claim 보조	patents, invention logs
Business	IR / Grant Agent	투자/정부과제/외부 자료 생성	source-grounded claims
People	Onboarding Agent	팀원 교육과 내부 규칙 전달	playbooks, SOPs, repo docs

6. 30-60-90 실행 계획

이 계획은 '가능한 빨리 DB화하고 싶다'는 요구와 '품질/컴플라이언스는 절대 희생하지 않는다'는 요구를 동시에 만족시키는 최소 고품질 실행안이다.

기간	목표	구체 작업	산출물
0-30일	RAG Core 안정화	registry schema lock, approved-only ingestion, retrieval eval, answer contract, token v1.4 compression	V1.x stable RAG core, eval report, source registry
31-60일	외부 Intelligence Layer	Deep Research 후보 수집, Literature/Benchmark agent, competitor registry, license triage	reviewable source packs, benchmark playbook, literature summaries
61-90일	R&D Design Layer	Hypothesis agent, experiment design schema, biosafety gate, DBTL package template	falsifiable hypothesis package, controlled experiment plan, DBTL tracker
90일 이후	DBTL/Data Flywheel	failure analysis, active learning, proprietary dataset curation, biofoundry ops integration	cycle-level learning system, dataset versions, model-readiness report

가장 중요한 1개 원칙

모든 Phase는 '자료를 더 많이 넣기'가 아니라 '다음 의사결정을 더 정확하게 만들기' 위해 존재한다. 따라서 각 작업의 끝에는 반드시 eval, evidence log, next-cycle recommendation이 있어야 한다.

7. 공개 벤치마크 참고자료

아래 자료는 본 문서의 벤치마크 원칙을 만들 때 참고한 공개 자료다. 내부 구현 시에는 각 source를 registry에 따로 등록하고, license/commercial-use/model-training 가능성을 별도로 검토해야 한다.

ID	Source	Why it matters	URL
S1	OpenAI Developers – Agents SDK	Agents are applications that plan, call tools, collaborate across specialists, and keep state.	https://developers.openai.com/api/docs/guides/agents
S2	OpenAI Developers – Safety in building agents	Tool approvals, guardrails, trace graders, evals, structured isolation.	https://developers.openai.com/api/docs/guides/agent-builder-safety
S3	Anthropic – Building effective agents	Successful implementations use simple, composable patterns: workflows and agents are distinct.	https://www.anthropic.com/engineering/building-effective-agents
S4	Anthropic – Responsible Scaling Policy	Capability thresholds, ASL safeguards, CBRN-related risk handling, defense-in-depth.	https://www.anthropic.com/responsible-scaling-policy
S5	LangGraph overview	Persistence, human-in-the-loop, memory, debugging, production deployment for stateful workflows.	https://docs.langchain.com/oss/python/langgraph/overview
S6	Benchling R&D Cloud	Registration, tracking, connected searchable platform, compliant scalable workflows.	https://www.benchling.com/resources/the-benchling-rd-cloud
S7	Ginkgo Bioworks – Autonomous Labs	Labwork bottleneck, autonomous labs, cloud lab, scientist focus on insight and judgment.	https://www.ginkgo.bio/
S8	Recursion – Technology / Recursion OS	Proprietary data, automated high-throughput labs, maps of biology/chemistry, active learning.	https://www.recursion.com/platform
S9	NCBI Bookshelf – DBTL and AI in synthetic biology	DBTL as iterative design, build, test, learn loop with automated labs and AI acceleration.	https://www.ncbi.nlm.nih.gov/books/NBK614601/
S10	Isomorphic Labs	Frontier AI, predictive and generative models, digital biology, AlphaFold lineage.	https://www.isomorphiclabs.com/

Final Bottom Line

Asperitas의 최상위 AI 프로세스는 OpenAI식 eval/trace/guardrail, Anthropic식 simple composable workflow와 safety threshold, Recursion식 proprietary data flywheel, Ginkgo식 autonomous lab/foundry, Benchling식 R&D registry, LangGraph식 stateful orchestration, DBTL식 과학 반복 루프를 결합한 A-GRID다. 이 구조로 가야 RAG가 단순 챗봇이 아니라 Biology Foundation Model을 향한 운영체계가 된다.